Intelli Service API Documentation

Subscription Status Integration Guide

1. Overview

This document describes the Intelli API endpoint required to retrieve a user's subscription status for a service. OTP authentication is not part of this integration flow.

The API flow is typically:

1. Collect or obtain the user's MSISDN.
2. Fetch the user's subscription status.
3. Grant access when an active subscription exists, or process the client_action instructions when it does not.

2. Base URL

```
https://api.intellihq.net
```

3. Service Context

The service identifier used by the endpoint URLs in this guide is:

```
82
```

All service-scoped endpoints in this guide use:

```
/api/v1/service/82/
```

4. Data Format

All API responses use JSON.

Recommended request header:

```
Accept: application/json
```

5. MSISDN Format

The msisdn field represents the user's phone number.

Recommended format:

```
2348012345678
```

The MSISDN should include the country code and should not include the leading 0, +, spaces, or special characters.

6. Endpoint Summary

Endpoint	Method	Purpose
/api/v1/service/82/subscription/status/	GET	Fetch user subscription status

7. Fetch User Subscription Status

This endpoint retrieves the subscription status of a user using their MSISDN.

Endpoint

```
GET /api/v1/service/82/subscription/status/?msisdn={msisdn}
```

Full URL

```
https://api.intellihq.net/api/v1/service/82/subscription/status/?msisdn={user_msisdn}
```

Query Parameters

Parameter	Type	Required	Description
msisdn	string	Yes	User's phone number in international format.

Example Request

```
curl -X GET "https://api.intellihq.net/api/v1/service/82/subscription/status/?msisdn=2348012345678" \
  -H "Accept: application/json"
```

7.1 Active Subscription Response

HTTP Status: 200 OK

```
{
  "success": true,
  "data": {
    "service_id": 42,
    "msisdn": "2348012345678",
    "has_any_subscription": true,
    "has_active_subscription": true,
    "active_subscription": {
      "subscription_id": 12345,
      "sub_status": "active",
      "sub_active": true,
      "telco": "MTN",
      "traffic_source": "MOBPLUS",
      "active_product_id": 10,
      "auto_renewal": true,
      "starts_date": "2025-11-20T10:00:00+01:00",
      "ends_date": "2025-11-27T10:00:00+01:00"
    }
  }
}
```

7.2 No Active Subscription Response

HTTP Status: 200 OK

```
{
  "success": true,
  "data": {
    "service_id": 42,
    "msisdn": "2348012345678",
    "has_any_subscription": false,
```

```
    "has_active_subscription": false,
    "active_subscription": null,
    "billing_records": [],
    "client_action": [
      {
        "action": "prompt",
        "instruction": "Send SUB keyword to 1234"
      },
      {
        "action": "redirect",
        "redirection_url": "https://example.com/subscribe?trfsrc=organic&trxId=ABC123DEF456GHI789",
        "campaign_urls": [
          {
            "antifraud": "envina",
            "telco": "MTN",
            "type": "WEB",
            "url": "https://example.com/subscribe",
            "product_id": "P10"
          },
          {
            "antifraud": "mfilter",
            "telco": "GLO",
            "type": "WEB",
            "url": "https://example.com/subscribe-alt",
            "product_id": "P10"
          }
        ]
      }
    ]
  }
}
```

Response Fields

Field	Type	Description
success	boolean	Indicates whether the request was processed successfully.
data	object	Subscription status data.
data.service_id	integer	ID of the service returned by the API.
data.msisdn	string	User's phone number.
data.has_any_subscription	boolean	Indicates whether the user has ever had a subscription.
data.has_active_subscription	boolean	Indicates whether the user currently has an active subscription.
data.active_subscription	object/null	Active subscription details if available; otherwise null.
data.billing_records	array	Billing records related to the user, if available.
data.client_action	array	Ordered client actions returned when the user has no active subscription.
data.client_action[].action	string	Action to perform, e.g. prompt or redirect.
data.client_action[].instruction	string	User-facing instruction for a prompt action, when present.
data.client_action[].redirection_url	string	Primary redirect URL for a redirect action, when present.
data.client_action[].campaign_urls	array	Campaign URL options associated with the redirect action, when present.

Possible Error Responses

Missing MSISDN

```
400 Bad Request

{
  "success": false,
  "message": "MSISDN is required"
}
```

Invalid MSISDN

```
400 Bad Request

{
  "success": false,
  "message": "Invalid MSISDN"
}
```

8. Recommended Client Flow

Step 1: Obtain the User's MSISDN

Obtain and normalize the user's phone number before making the subscription-status request.

```
{ "msisdn": "2348012345678" }
```

Step 2: Fetch Subscription Status

```
GET /api/v1/service/82/subscription/status/?msisdn=2348012345678
```

Step 3: Route the User Based on Subscription Status

If has_active_subscription is true, allow the user to access the service.

```
{
  "has_active_subscription": true,
  "active_subscription": {
    "sub_status": "active"
  }
}
```

If has_active_subscription is false, inspect client_action. It is an array and may contain more than one instruction. Process the actions relevant to your client experience.

- For action = prompt, display the supplied instruction to the user.
- For action = redirect, use the supplied redirection_url. campaign_urls may also contain telco/antifraud-specific campaign destinations.

Example:

```
{
  "client_action": [
    {
      "action": "prompt",
      "instruction": "Send SUB keyword to 1234"
    },
    {
      "action": "redirect",
      "redirection_url": "https://example.com/subscribe?trfsrc=organic&trxId=ABC123DEF456GHI789",
      "campaign_urls": [...]
    }
  ]
}
```

9. JavaScript Integration Examples

9.1 Fetch Subscription Status

```
async function fetchSubscriptionStatus(msisdn) {
  const url = new URL(
    "https://api.intellihq.net/api/v1/service/82/subscription/status/"
  );
  url.searchParams.set("msisdn", msisdn);

  const response = await fetch(url.toString(), {
    method: "GET",
    headers: { "Accept": "application/json" }
  });

  const data = await response.json();

  if (!response.ok || !data.success) {
    throw new Error(data.message || "Unable to fetch subscription status");
  }

  return data;
}
```

9.2 Example Client Routing Logic

```
async function handleUserAccess(msisdn) {
  const subscriptionStatus = await fetchSubscriptionStatus(msisdn);
  const statusData = subscriptionStatus.data;

  if (statusData.has_active_subscription) {
    return {
      action: "grant_access",
      subscription: statusData.active_subscription
    };
  }

  const actions = Array.isArray(statusData.client_action)
    ? statusData.client_action
    : [];

  const promptAction = actions.find(item => item.action === "prompt");
  const redirectAction = actions.find(item => item.action === "redirect");

  if (promptAction?.instruction) {
    // Render this instruction in the client UI.
    console.info(promptAction.instruction);
  }

  if (redirectAction?.redirection_url) {
    window.location.href = redirectAction.redirection_url;
    return;
  }

  return {
    action: "deny_access",
    message: "No active subscription found"
  };
}
```

10. Status Code Guide

Status Code	Meaning	Description
200	OK	Request was successful.
400	Bad Request	Invalid request, missing MSISDN, or invalid MSISDN.
404	Not Found	Endpoint or resource not found.

405	Method Not Allowed	Wrong HTTP method used for the endpoint.
500	Internal Server Error	Unexpected server error.

11. Error Handling Recommendation

Client applications should not rely only on the HTTP status code. They should also check the success field in the response body.

```
if (!response.ok || data.success === false) {
  showError(data.message || "Something went wrong. Please try again.");
}
```

API Error	Suggested Client Message
MSISDN is required	Please enter your phone number.
Invalid MSISDN	Please enter a valid phone number.

12. Security Recommendations

- Normalize MSISDN before sending it to the API.
- Use HTTPS for all API requests.
- Avoid exposing sensitive user or subscription data in client-side logs.
- Validate API responses before acting on returned URLs or instructions.

13. Notes for Frontend Developers

- Always normalize the user's phone number before making API calls.
- Always check both response.ok and the response body's success field.
- Treat active_subscription as nullable.
- Treat client_action as an array, not a single object.
- Handle prompt and redirect actions independently when both are returned.
- Do not hardcode the subscription URL when the API provides redirection_url.
- Use loading and error states while fetching subscription status.

14. Example End-to-End Flow

```
async function completeSubscriptionFlow(msisdn) {
  const result = await fetchSubscriptionStatus(msisdn);
  const userData = result.data;

  if (userData.has_active_subscription) {
    return {
      subscribed: true,
      subscription: userData.active_subscription
    };
  }

  const actions = Array.isArray(userData.client_action)
    ? userData.client_action
    : [];

  return {
    subscribed: false,
    actions
  };
}
```

15. Changelog

Version	Date	Description
1.1	2026-09-23	Removed OTP send/verify documentation and OTP-based flow; updated service endpoint URLs to service ID 82; updated no-active-subscription response and client_action handling; revised integration examples.
1.0	2026-05-12	Initial API documentation for OTP authentication and subscription status endpoints.